# Risk Documentation

## Smart House System

## Revision History

| Date | Version | Description | Author |
|---|---|---|---|
| 2026-02-08 | 1.0 | Initial potential risks for the Smart House System | Daniel Marcarini |
| 2026-02-24 | 1.1 | AI Risk Extension (R10-R12) | Md Abu Taher |
| | | AI Risk Extension (R13-R14) | Anitta Antony |
| 2026-02-28 | 1.1 | Smart Home code | Daniel Marcarini |
| 2026-02-28 | 1.1 | Speech Risk (R15) | Daniel Holderik |
| 2026-03-28 | 1.2 | Updated with Web and Firebase | Md Abu Taher<br>Anitta Antony |
| 2026-03-28 | 1.2 | Testing | Daniel Marcarini |
| 2026-04-22 | 1.3 | Updated risk with certain values | Anitta Antony<br>Md Abu Taher |

## Risk List

| Risk Description | Priority | Progress |
|---|---|---|
| R1. Integration Complexity Between System Components | High | |
| R2. Scalability and Performance Limitations | Medium-High | |
| R3. Incomplete or Over-Ambitious Innovation Scope | High | |
| R4. Unclear Requirements or Changing Requirements | Medium | |
| R5. Unauthorized Access or Security Vulnerabilities | High | |
| R6. Data Privacy Concerns | Medium-High | |
| R7. Uneven Work Distribution | Medium | |
| R8. Time Constraints and Deadline Pressure | Critical | |
| R9. Insufficient Testing | High | |
| R10. AI misinterpretation of User commands | High | Done |
| R11. Delay in AI Response Time | Medium- High | In progress |

| | | |
|---|---|---|
| R12. Over-Reliance on AI Decision Logic | Medium | Done |
| R13. AI Service Dependency and External AI Service Failure | High | In progress |
| R14. Incorrect AI response in safety-critical Scenarios | High | Done |
| R15. Dependency on external speech recognition library | High | |
| R16. insufficient verification of web application and device-control integration | High | |

# Risk Handling Plans

## Contents


Risk Documentation....1
Smart House System....1
Risks – SG4 Innovation....2
1 Introduction....2
Technical Risks....3
Risk 1: Integration Complexity Between System Components....3
Risk 2: Scalability and Performance Limitations....3
Risk 3: Incomplete or Over-Ambitious Innovation Scope....4
Risk 4: Unclear Requirements or Changing Requirements....4
Security and Privacy Risks....4
Risk 5: Unauthorized Access or Security Vulnerabilities....4
Risk 6: Data Privacy Concerns....5
Project Management Risks....5
Risk 7: Uneven Work Distribution....5
Risk 8: Time Constraints and Deadline Pressure....5
Quality Risks....6
Risk 9: Insufficient Testing....6
Summary....6

# Risks – SG4 Innovation

## 1 Introduction

This section identifies and evaluates the key risks associated with the SmartHouse project, particularly those related to innovation, system complexity, and technical integration. Since SG4 focuses on innovation and refinement, several technical and project-related uncertainties must be considered to ensure successful implementation and delivery.
Each risk is evaluated in terms of:

- **Description**
- **Likelihood** (Low / Medium / High)
- **Impact** (Low / Medium / High)
- **Mitigation Strategy**

# Technical Risks

## Risk 1: Integration Complexity Between System Components

**Description:**
The SmartHouse system consists of multiple interacting components (e.g., sensors, controllers, user interface, backend logic, possible IoT simulation). Integration between these modules may introduce unexpected behavior, compatibility issues, or architectural inconsistencies.

**Likelihood:** Medium
**Impact:** High

**Mitigation Strategy:**
- Use a modular architecture with clearly defined interfaces.
- Apply early integration testing instead of postponing integration until late stages.
- Follow consistent design principles and coding standards.
- Conduct peer code reviews.

## Risk 2: Scalability and Performance Limitations

**Description:**
Innovative features (e.g., automation rules, real-time monitoring, event handling) may increase system complexity. If not properly designed, the system may experience performance degradation as the number of devices or rules increases.

**Likelihood:** Medium
**Impact:** Medium to High

**Mitigation Strategy:**
- Use efficient data structures and algorithms.
- Perform early performance testing with simulated load.
- Avoid unnecessary polling or blocking operations.
- Refactor performance-critical sections when identified.

## Risk 3: Incomplete or Over-Ambitious Innovation Scope

**Description:**
Since SG4 emphasizes innovation, there is a risk that the team attempts to implement overly complex or experimental features that exceed the time or technical capacity of the group.

**Likelihood:** Medium
**Impact:** High

**Mitigation Strategy:**
- Prioritize core functionality before innovative extensions.
- Define a Minimum Viable Product (MVP).
- Time-box experimental features.
- Regularly evaluate progress against deadlines.

## Risk 4: Unclear Requirements or Changing Requirements

**Description:**
Smart house systems often involve evolving user expectations. Changes in requirements during development may cause rework and architectural instability.

**Likelihood:** Medium
**Impact:** Medium

**Mitigation Strategy:**
- Clearly document functional and non-functional requirements.

- Use version control for requirement changes.
- Apply incremental development with frequent feedback.
- Maintain traceability between requirements and implementation.

# Security and Privacy Risks

## Risk 5: Unauthorized Access or Security Vulnerabilities

**Description:**
A smart house system may simulate or implement authentication, device control, and user data handling. Weak security design may lead to vulnerabilities such as unauthorized access or manipulation of devices.

**Likelihood:** Medium
**Impact:** High

**Mitigation Strategy:**
- Implement proper authentication and authorization mechanisms.
- Validate all user inputs.
- Avoid hard-coded credentials.
- Follow secure coding practices.

## Risk 6: Data Privacy Concerns

**Description:**
If the system stores user preferences, logs, or sensor data, there is a risk of mishandling sensitive information.

**Likelihood:** Low to Medium
**Impact:** High

**Mitigation Strategy:**
- Minimize stored personal data.
- Apply encryption if sensitive data is stored.
- Clearly define data storage policies.
- Restrict access to stored data.

# Project Management Risks

## Risk 7: Uneven Work Distribution

**Description:**
In group-based software engineering projects, some members may become overloaded while others contribute less, affecting quality and deadlines.

**Likelihood:** Medium
**Impact:** Medium

**Mitigation Strategy:**
- Clearly define responsibilities.
- Use task tracking tools.
- Hold regular team meetings.
- Ensure transparency in workload distribution.

### Risk 8: Time Constraints and Deadline Pressure

**Description:**
Innovation often requires experimentation and iteration, which may conflict with fixed academic deadlines.

**Likelihood:** High
**Impact:** High

**Mitigation Strategy:**
- Create a detailed project timeline.
- Prioritize essential features.
- Reserve buffer time for testing and debugging.
- Monitor progress weekly.

## Quality Risks

### Risk 9: Insufficient Testing

**Description:**
Due to focus on innovation and feature development, testing may be deprioritized, resulting in unstable functionality.

**Likelihood:** Medium
**Impact:** High

**Mitigation Strategy:**
- Implement unit testing for core components.
- Perform integration testing before delivery.
- Use test cases derived from requirements.
- Automate testing where possible.

## AI Risks

### Risk 10: AI misinterpretation of User commands

**Description:**
Since the system relies on natural language processing, the AI may incorrectly interpret ambiguous or complex user commands. This may result in unintended device actions, such as controlling the wrong device or executing an incorrect command. Misinterpretation is identified when the executed action does not match the user's intended request.

**Likelihood:** Medium

**Impact:** Medium to High

**Mitigation Strategy:**

- Use structured prompting to constrain AI output.
- Display AI response before executing any physical action.
- Mark confirmation and action validation as a future enhancement for the device-control layer.

## Risk 11: Delay in AI Response Time

**Description:**
AI processing requires communication with an external AI service, which may introduce latency. A delay is considered problematic if the system does not provide feedback quickly or if the response takes too long, causing the user to think the system is not working. According to usability guidelines(Nielsen,1993), system should provide feedback within about 1 second to maintain user attention, while delays longer than several seconds reduce usability. Based on this, response times longer than approximately 5 seconds are considered slow for this system.
**Likelihood:** Medium
**Impact:** Medium to high
**Mitigation Strategy:**

- Displays a loading indicator within 1 second after sending a request.
- Prevent duplicate requests is in progress.
- Aim to keep most AI response withing 5 seconds.
- Backend performance optimization or caching maybe applied if needed

## R12:Over-Reliance on AI Decision Logic

**Description:**
Relying heavily on AI decision-making may reduce system predictability. AI-generated outputs may not always align with strict safety or operational requirements. AI response time and behaviour should not affect safety-critical operations.

**Likelihood:** Medium
**Impact:** Medium

**Mitigation Strategy:**

- Ensure safety-critical actions are handled outside the AI module.
- Keep control logic within the device or system layer, independent of AI processing

## R13: AI Service Dependency and External AI Service Failure

**Description:**

The system depends on an external AI service. If the service becomes unavailable, unstable, or slow to respond, AI-based command processing may fail or be delayed. A response is considered slow if it takes longer than approximately 5 seconds, as delays beyond this point may negatively affect user experience. According to usability guidelines, systems should provide feedback within about 1 second to keep the user informed, while longer delays require visible indication of processing (Nielsen, 1993).

Based on this, the system shall display feedback within 1 second after a request is sent, and AI responses should normally be received within 5 seconds. If a response is not received within approximately 5–10 seconds, it is considered a failure from the user's perspective.

**Likelihood:** Medium
**Impact:** High

**Mitigation Strategy:**
- Display fallback response when the AI service fails or times out.
- Inform the user to check network or backend connectivity.
- Log failures for future analysis

## R14. Incorrect AI response in safety-critical Scenarios

**Description:**
If AI incorrectly handles commands related to safety (e.g., gas or fire conditions), it may provide misleading information or delay proper action. In safety-critical situations, any delay caused by AI processing is unacceptable.

**Likelihood:** Low to medium
**Impact:** High

**Mitigation Strategy:**
- Do not allow AI to control emergency response directly.
- Ensure safety actions are handled immediately by the sensor/device subsystem.
- Use AI only for guidance or notification messages, not for safety decisions.

## R15 – Dependency on external speech recognition library

**Description:**
The speech recognition feature depends on an external library and native device capabilities. Compatibility issues, library limitations, or platform restrictions may affect functionality.

**Mitigation Strategy:**
Use a supported and maintained library.
Test on multiple devices.
Provide fallback to text input if speech recognition fails.

### R16 - Insufficient verification of web application and device-control integration

**Description:**
There is a risk that the web application appears to function correctly at interface level while the underlying device-control flow behaves incorrectly when connected to Firebase and the device state backend. This may lead to incorrect updates, inconsistent displayed states, or unnoticed faults in integration between the web interface and the device-control logic.

**Probability:** Medium
**Impact:** High

**R16 – Browser compatibility and Web Speech API limitations**

**Description:**

The Web Speech API is not supported consistently across all browsers and devices. Differences in implementation or lack of support may affect speech recognition reliability.

Mitigation Strategy:

Test functionality across supported browsers.

Provide fallback to text input where speech is unavailable.

Inform users when speech recognition is not supported.

**Probability**: Low

**Impact** :High

**Handling Plan:**

- Perform manual integration testing on the implemented web application flows
- Verify that frontend actions result in correct Firestore updates
- Verify that Firestore state changes are reflected correctly in the hub interface
- Prioritize testing of login, signup, guest access, hub loading, and toggle actions
- Record defects and perform retesting after fixes
- Consider lightweight unit testing for frontend validation logic in a later step

## Summary

The SmartHouse project involves technical integration, innovative feature development, and collaborative software engineering. The most critical risks relate to integration complexity, time constraints, and over-ambitious innovation goals. By applying structured planning, modular design, continuous testing, and clear team coordination, these risks can be significantly reduced.

Proper risk management ensures that innovation enhances the project rather than jeopardizes delivery, quality, or maintainability.

# Daniel Marcarini - Smart Home code

| Risk Description | Priority |
|---|---|
| R1. Servo power overload and voltage drop | High |
| R2. Blocking code affecting emergency response | High |
| R3.Sensor false positives or missed detection | High |
| R4. State machine logic errors | Medium-High |
| R5. Hardware component failure | Medium |
| R6. Electrical wiring instability | Medium |
| R7. Limited hardware scalability | Medium |
| R8. Insufficient embedded testing | High |

## Risk Handling Plans

# 1 Introduction

This section identifies and evaluates the technical risks associated with the embedded Smart House implementation. Since the system relies on hardware integration, real-time monitoring, and state-based logic, both software and hardware risks must be considered.

Each risk is evaluated in terms of:

- Description
- Likelihood (Low / Medium / High)
- Impact (Low / Medium / High)
- Mitigation Strategy

# Technical Risks

### Risk 1: Servo Power Overload and Voltage Drop

Description:
Servo motors draw significantly more current than other components. Simultaneous activation of servos and alarm systems may cause voltage instability, unexpected resets, or system malfunction.

Likelihood: Medium
Impact: High

Mitigation Strategy:
- Avoid activating multiple high-current components simultaneously.
- Sequence actuator operations when possible.
- Ensure stable and sufficient power supply.
- Test system behavior under peak load conditions.

### Risk 2: Blocking Code Affecting Emergency Response

Description:
If blocking delays are used in the main loop, the system may become temporarily unresponsive. This may delay safety-critical responses such as gas alarm activation.

Likelihood: Medium
Impact: High

Mitigation Strategy:
- Use non-blocking timing mechanisms.
- Avoid delay-based logic in emergency procedures.
- Continuously test responsiveness under multiple simultaneous conditions.

### Risk 3: Sensor False Positives or Missed Detection

Description:
Gas and environmental sensors may produce unstable readings due to noise, environmental interference, or improper calibration. This may lead to false alarms or missed hazard detection.

Likelihood: Medium
Impact: High

Mitigation Strategy:
- Calibrate sensors properly.
- Use threshold validation and stable detection logic.

- Implement state flags to prevent repeated triggering.
- Test under various environmental conditions.

### Risk 4: State Machine Logic Errors

Description:
Incorrect state transitions may cause repeated emergency activation, inconsistent window states, or incorrect alarm behavior.

Likelihood: Low to Medium
Impact: High

Mitigation Strategy:
- Use clearly defined state variables.
- Validate transitions during testing.
- Document state logic clearly in design documentation.
- Perform scenario-based testing.

### Risk 5: Hardware Component Failure

Description:
Failure of a servo motor, sensor, LCD, or buzzer may reduce system functionality or compromise monitoring capabilities.

Likelihood: Medium
Impact: Medium to High

Mitigation Strategy:
- Isolate safety-critical logic from non-critical components.
- Validate hardware components before integration.
- Replace faulty components promptly.

### Risk 6: Electrical Wiring Instability

Description:
Loose wiring, poor grounding, or incorrect connections may cause unstable readings or system malfunction.

Likelihood: Medium
Impact: Medium

Mitigation Strategy:
- Use organized wiring layout.
- Secure physical connections.
- Test wiring integrity before deployment.

### Risk 7: Limited Hardware Scalability

Description:
The Arduino Uno platform has limited memory and processing power. Adding additional sensors or complex logic may exceed hardware limitations.

Likelihood: Medium
Impact: Medium

Mitigation Strategy:
- Optimize code efficiency.
- Avoid unnecessary memory usage.
- Plan hardware expansion carefully.

### Risk 8: Insufficient Embedded Testing

Description:
Embedded systems interact directly with hardware. Insufficient real-world testing may result in unstable or unsafe system behavior.

Likelihood: Medium
Impact: High

Mitigation Strategy:
- Test each subsystem independently.
- Perform full integration testing.
- Simulate emergency conditions during validation.
- Validate behavior under long-running operation.

REFERNCES

1. Nielsen, J. (1993). *Usability Engineering*. Morgan Kaufmann.